// BEFORE THE FIRST COMMAND

What is in a project?

The words and the folders every newcomer meets first



// 01 PROJECT BASICS

A project is a folder of many kinds of files

meridian-ai

```
meridian-ai/
├── backend/
│   └── rag/retrieval.py  # code
├── sample_docs/  # data
├── .env  # secret: never in Git
├── .env.example  # settings template
├── requirements.txt  # settings: the shopping list
├── .github/workflows/deploy.yml  # automation
├── README.md  # documents
└── .venv/  # generated: never edit
```

// SEVEN KINDS, SEVEN RULES

Code	Instructions the computer runs. Read it, change it carefully.
Settings	Values that change behaviour. Change values, not logic.
Secrets	Keys and passwords. Never put them in Git.
Data	Input files. Replace with your own.
Documents	Explanations for people. Keep them up to date.
Automation	Scripts a robot runs. Trigger it and let it run.
Generated	Made by tools. Never edit; not saved in Git.

// 01 PROJECT BASICS

Secrets and generated files stay out of Git

Secrets

An API key or a password. Once it is in Git it is on the internet for good. Keep it in `.env`.

Generated files

Folders such as `.venv/` that a tool can recreate. Saving them only adds clutter.

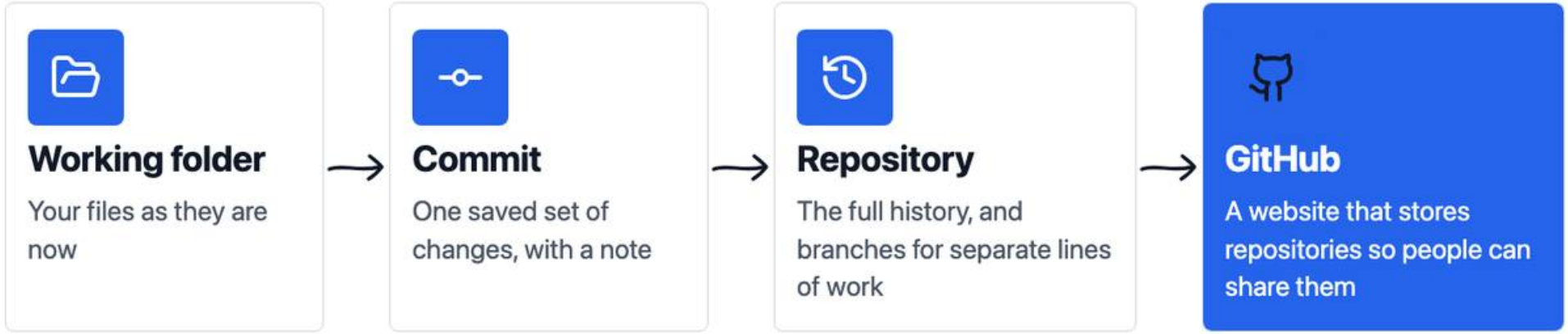
// ONE FILE LISTS THEM ALL

.gitignore

```
# .gitignore: what Git must not save
.env
.venv/
__pycache__/
node_modules/
logs/
.meridian/
```

> That is how a password avoids ending up on the internet by accident.

Git and GitHub in four words



the three commands you will see

```
$ git add -A
$ git commit -m "Project skeleton"
$ git push
```

fork = your own copy of someone else's repository

// 01 PROJECT BASICS

Frontend and backend, one conversation



the API: a menu of web addresses, each doing one job

// DATA TRAVELS AS JSON

one request, one reply

```
POST /api/rag/ask
{"query": "What is the FY2024 revenue?"}

# the reply
{"answer": "EUR 2.84 billion"}
```

JSON looks like a Python dictionary.

// 01 PROJECT BASICS

Python words you will hear all day

backend/rag/llm.py (excerpt)

```python
# backend/rag/llm.py    3
from langchain_google_genai import ChatGoogleGenerativeAI  1
from config.settings import settings  2

# rag/ is a folder with an __init__.py file  4
```

1 Dependency
Code someone else wrote, installed from the internet. This one comes from the langchain-google-genai package.

2 Import
Using code from another file of our own: here, config/settings.py.

3 Module
One Python file. This one is llm.py.

4 Package
A folder of modules that has an __init__.py file. This one is rag/.

Dependencies and pinned versions

requirements.txt (excerpt)

```
# pinned: an exact version
fastapi==0.135.1 (1)
langchain==1.2.11
pydantic==2.12.5
pypdf==6.7.5

# not pinned: any version
fastapi (3)
```

1 **Pinned version**
An exact version number. Everyone who installs the file gets the same code.

2 **Why pin**
AI libraries change fast and can break each other. Pinning means it works the same on every computer.

3 **Without a pin**
fastapi alone means any version: it works on your laptop today and fails on a colleague's tomorrow.

// 01 PROJECT BASICS

Settings stay outside the code

.env.example

A template: names, no real values. Saved in Git.

→

Copy it

Make your own file called .env

→

.env

Your real values. Never saved in Git.

→

The program

Reads them as environment variables

.env.example (first lines)

```
APP_MODE=local
ENVIRONMENT=local
GOOGLE_API_KEY=your-gemini-api-key
```

// 01 PROJECT BASICS

One file, one job

PROBLEM WITH ONE BIG FILE	HOW SPLITTING SOLVES IT
Hard to find things	The name tells you where to look, for example agent/prompts.py.
One change breaks another	A change stays inside its own file.
Hard to test	Parts can be tested alone, without the internet.
Copy-pasted code drifts apart	One function is written once and reused.
Several people clash	Different people edit different files.
Scary to read	Read one small file at a time.

// SUMMARY

The basics in five lines

1. **A project holds many kinds of files.**
   Code, settings, secrets, data, documents, automation, generated.

2. **Secrets and generated files stay out of Git.**
   The .gitignore file is the list.

3. **The frontend is the screen; the backend is the engine.**
   They talk through an API, in JSON.

4. **Dependencies are pinned to exact versions.**
   So it behaves the same on every computer.

5. **One file, one job.**
   That is how the code stays easy to find and change.

// GETTING READY

Your tools, and the new project

Python 3.12, uv, Git and an editor, then one command to begin



// 02 YOUR TOOLS

What we need on your machine



Python 3.12

The language. uv installs the right version for us.

needed now



uv

One tool for Python versions, environments and packages.

needed now



Git

Saves the history of the project and shares it.

needed now



An editor

Visual Studio Code, or any editor you like.

needed now



Node.js 20 or newer

Only for the web screen, later in the series.

later



A Gemini API key

Free from Google AI Studio. We create it with the settings.

later

// 02 YOUR TOOLS

The older routine

pip and venv

```
$ python3.12 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt

# forget to activate in a new terminal
```

Easy to forget a step

A new terminal, and you are outside the environment.

Several tools to learn

Python, venv and pip, each with its own habits.

With uv

uv

```
$ uv init --python 3.12
$ uv add -r requirements.txt
$ uv run python app.py

# nothing to activate, ever
```

One tool

Python version, environment, packages and running.

Same result, fewer steps

uv run uses the right environment automatically.

// 02 YOUR TOOLS

Install uv



macOS and Linux (terminal)

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```



Windows (PowerShell) (PowerShell)

```
$ powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```



Or a package manager (alternatives)

```
$ brew install uv                      # macOS
  winget install --id=astral-sh.uv -e   # Windows
```

then close and reopen the terminal, and check:

check

```
$ uv --version
uv 0.8.22 (your number will differ)
```

Install Python 3.12 with uv

install Python

```
$ uv python install 3.12
Installed Python 3.12.11 in 99ms
 + cpython-3.12.11-macos-aarch64-none (python3.12)
```

on this laptop the default python3 is 3.14

1. **Python 3.12**
   The packages are pinned and tested on 3.12. A newer default Python can break the pins, so we ask for 3.12 explicitly.

2. **Side by side**
   uv keeps its own Pythons. It does not touch the Python your operating system uses.

3. **Do it once**
   After this, every project that asks for 3.12 reuses it.

// 02 YOUR TOOLS

Create the project

create

```
$ mkdir meridian-ai
$ cd meridian-ai
$ uv init --python 3.12 --no-readme
Initialized project `meridian-ai`
```

> --no-readme keeps the folder lean. We use the README from the reference repository.

after uv init

```
.../meridian-ai/
├── .git/  # uv also created a Git repository
├── .gitignore
├── .python-version
├── main.py  # a placeholder
└── pyproject.toml  # the project's identity card
```

What uv created

pyproject.toml and .python-version

```
# pyproject.toml
[project]
name = "meridian-ai"  1
version = "0.1.0"
requires-python = ">=3.12"  2
dependencies = []  3

# .python-version
3.12  4
```

1 Name
The project's name. The rest of the file describes it.

2 Python rule
This project needs Python 3.12 or newer.

3 Dependencies
Empty for now. The next chapter fills this list.

4 The pin
.python-version says exactly 3.12, so uv always picks it.

// 02 YOUR TOOLS

Bring in the starter files

run from inside meridian-ai

```
$ git clone https://github.com/mayank953/meridian-ai-learner.git ../meridian-reference
$ cp ../meridian-reference/requirements.txt .
$ cp ../meridian-reference/requirements-dev.txt .
$ cp ../meridian-reference/.env.example .
$ cp -R ../meridian-reference/sample_docs .
$ rm main.py
```

> Same names, different folder: the reference repository stays next to our project, and we copy from it.

Copy now

requirements.txt, requirements-dev.txt, .env.example, sample_docs/

Type ourselves

Everything inside backend/, one file at a time.

Copy later

frontend/, the ready-made screen.

// 02 YOUR TOOLS

Check your starting point

checkpoint

```
$ ls -A
.env.example  .git  .gitignore  .python-version
pyproject.toml  requirements-dev.txt  requirements.txt
sample_docs
```

Windows: dir -Force

Eight entries
If yours matches, you are ready for the packages.

Something missing?
Run the copy commands again from inside meridian-ai.

No .git?
Run git init once.

// SUMMARY

Tools and project in five lines

1 **Four tools are needed now.**
Python 3.12, uv, Git and an editor.

2 **uv replaces venv and pip.**
One tool, nothing to activate.

3 **uv python install 3.12**
A pinned Python that does not touch your system.

4 **uv init --python 3.12 --no-readme**
Creates the project and a Git repository.

5 **Copy the starter files, type the backend.**
The reference repository stays beside our project.

// THE REQUIREMENT FILES

Packages and the environment

What the requirement files say, and how uv installs them



// 03 PACKAGES

Two files, two jobs

the two requirement files

```
# requirements.txt: what the app needs to run
fastapi==0.135.1
uvicorn==0.41.0
...
pypdf==6.7.5

# requirements-dev.txt: plus tools for testing
-r requirements.txt
pytest==8.4.2
httpx==0.28.1
```

requirements.txt
19 packages the app needs when it runs.

requirements-dev.txt
Everything above, plus pytest and httpx for the tests.

Why two?
The live server should not carry test tools.

// 03 PACKAGES

Reading requirements.txt: the web service

requirements.txt (1 of 3)

```
fastapi==0.135.1 1
uvicorn==0.41.0 1
python-multipart==0.0.22 2
pydantic==2.12.5 3
pydantic-settings==2.15.0 3
structlog==26.1.0 4
requests==2.34.2 5
```

1. **fastapi, uvicorn**
   The web server and the framework that turns web requests into Python function calls.
2. **python-multipart**
   Lets the server receive uploaded files, such as PDFs.
3. **pydantic, pydantic-settings**
   Checks the shape of requests, and reads settings from .env.
4. **structlog**
   Writes logs as structured JSON lines.
5. **requests**
   Makes the live exchange-rate call.

// 03 PACKAGES

Reading requirements.txt: the AI toolkit

requirements.txt (2 of 3)

```
langchain==1.2.11 1
langchain-core==1.2.18 2
langchain-community==0.4.1 3
langchain-text-splitters==1.1.1 4
langchain-google-genai==4.2.1 5
langgraph==1.1.6 6
```

1. **langchain**
   Agents and middleware: create_agent lives here.
2. **langchain-core**
   Prompts, tools and the pipe syntax for chains.
3. **langchain-community**
   The PDF loader.
4. **langchain-text-splitters**
   Cuts documents into pieces.
5. **langchain-google-genai**
   Gemini for chat and for embeddings.
6. **langgraph**
   The engine under create_agent. We write no graph code.

// 03 PACKAGES

Reading requirements.txt: storage and documents

requirements.txt (3 of 3)

```
langchain-chroma==1.1.0 1
chromadb==1.5.9 1
langchain-google-vertexai==3.2.2 2
google-cloud-aiplatform==1.141.0 2
google-cloud-storage==3.9.0 3
pypdf==6.7.5 4
```

1. **chromadb, langchain-chroma**
   Local mode: a vector database saved in a folder on your disk.

2. **langchain-google-vertexai, google-cloud-aiplatform**
   Cloud mode: Vertex AI Vector Search.

3. **google-cloud-storage**
   Cloud mode: a Cloud Storage bucket for the PDFs and logs.

4. **pypdf**
   Reads the text out of PDF files.

// 03 PACKAGES

Install with uv add

install

```
$ uv add -r requirements.txt
Resolved 157 packages in 1.13s
Installed 154 packages in 580ms
 + fastapi==0.135.1
 + langchain==1.2.11
 + ... and 150 more
```

> The first run downloads the packages and takes about a minute. The numbers and times on your screen will differ.

1. **Reads the file**
   19 packages, each pinned.
2. **Resolves**
   157 packages in total: yours bring their own dependencies.
3. **Installs**
   Into a private environment folder, .venv.
4. **Records**
   Updates pyproject.toml and writes uv.lock.

// 03 PACKAGES

What changed in your folder

// BEFORE

before

```
meridian-ai/
├── pyproject.toml  # dependencies = []
├── .python-version
└── requirements.txt
```

// AFTER UV ADD

after

```
meridian-ai/
├── pyproject.toml  # 19 packages, pinned
├── .python-version
├── requirements.txt
├── uv.lock  # exactly what was installed
└── .venv/  # the private environment
```



// 03 PACKAGES

Three things to know

COMMIT IT



pyproject.toml

What we asked for: 19 packages, each pinned. The project's shopping list.

COMMIT IT



uv.lock

Exactly what was installed: every one of the 157 packages and its version. About 4,000 lines; never edit by hand.

NEVER COMMIT



.venv/

The private environment, around 590 MB. Delete it any time and `uv sync` recreates it from the lock file.

Dev tools: pytest and httpx

add the test tools

```
$ uv add --dev pytest==8.4.2 httpx==0.28.1
```

pyproject.toml (new section)

```
[dependency-groups]
dev = [
    "httpx==0.28.1",
    "pytest==8.4.2",
]
```

pytest

Runs our 17 automated tests. We build them near the end.

httpx

Lets the tests call the web app without starting a server.

A separate group

Dev tools are listed apart so the live app does not carry them.

// 03 PACKAGES

Run code inside the environment

check it works

```
$ uv run python -c "import sys, fastapi, langchain;
print(sys.version.split()[0], fastapi.__version__, langchain.__version__)"
3.12.11 0.135.1 1.2.11
```

> Plain python would not see these packages. uv run does: it uses .venv automatically.

uv run

Runs a command inside the project's environment. Nothing to activate.

From any folder

It finds the project by looking upwards for pyproject.toml.

Read the numbers

Python 3.12, FastAPI 0.135.1, LangChain 1.2.11: the pins worked.

If something goes wrong

WHAT YOU SEE	LIKELY CAUSE	WHAT TO DO
uv: command not found	The terminal does not know uv yet.	Close and reopen the terminal. If it persists, run the installer again.
No module named 'fastapi'	You ran plain python, outside the environment.	Use uv run python instead.
A Python version error	The project did not get Python 3.12.	Run uv python install 3.12, then check .python-version says 3.12.
The first install is slow	The AI packages are large.	Wait. The next install uses the cache and takes seconds.
A download fails	A network or proxy problem.	Check the connection, then run the same command again.

// SUMMARY

Packages and environment in five lines

1. **requirements.txt is the shopping list.**
   19 packages, exact versions, in three groups.

2. **requirements-dev.txt adds the test tools.**
   Kept apart from the live app.

3. **uv add -r requirements.txt installs it all.**
   157 packages in a private .venv.

4. **uv.lock records exactly what was installed.**
   Commit it; never commit .venv.

5. **uv run runs code inside the environment.**
   Nothing to activate.

// WHERE EVERYTHING WILL LIVE

The folder structure: one file, one job

Every piece of code has a home before we write a single line of it



// 04 STRUCTURE

Where would you put it?

Answer first, then check.

QUESTION	YOUR ANSWER
Where does the tax prompt go?	agent/prompts.py
Where does the Gemini call go?	rag/llm.py
Where does a web address go?	api/endpoints.py
Where do settings come from?	config/settings.py
Where is the diary (the log)?	logger/

// 04 STRUCTURE

The map of backend/

api

```
api/
├── main.py  # creates the app, plugs everything in
├── endpoints.py  # every web address
└── schemas.py  # shapes of requests and replies
```

agent

```
agent/
├── tools.py  # five tools agents can use
├── prompts.py  # the job descriptions
├── middleware.py  # guard-rails
└── agents.py  # the team and the supervisor
```

rag

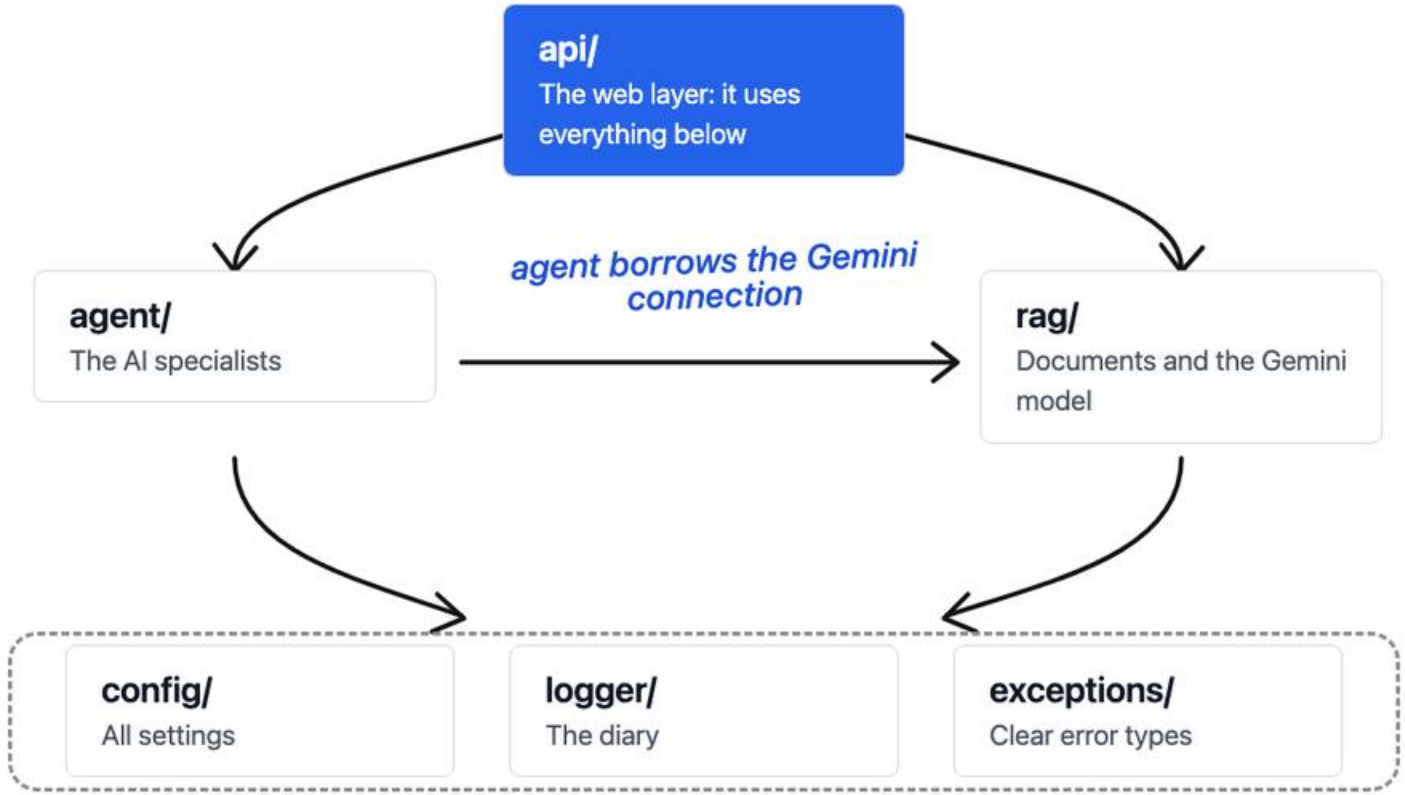
```
rag/
├── llm.py  # the Gemini chat model
├── embeddings.py  # text to numbers
├── vector_store.py  # where the numbers live
├── document_store.py  # keeps the original PDFs
├── data_ingestion.py  # PDF to pieces to stored
└── retrieval.py  # question to answer
```

everything else

```
backend/
├── config/settings.py  # all settings, one place
├── logger/custom_logger.py  # the diary
├── exceptions/__init__.py  # clear error types
└── tests/  # 3 files, 17 tests
```

Who may use whom

arrows point down only

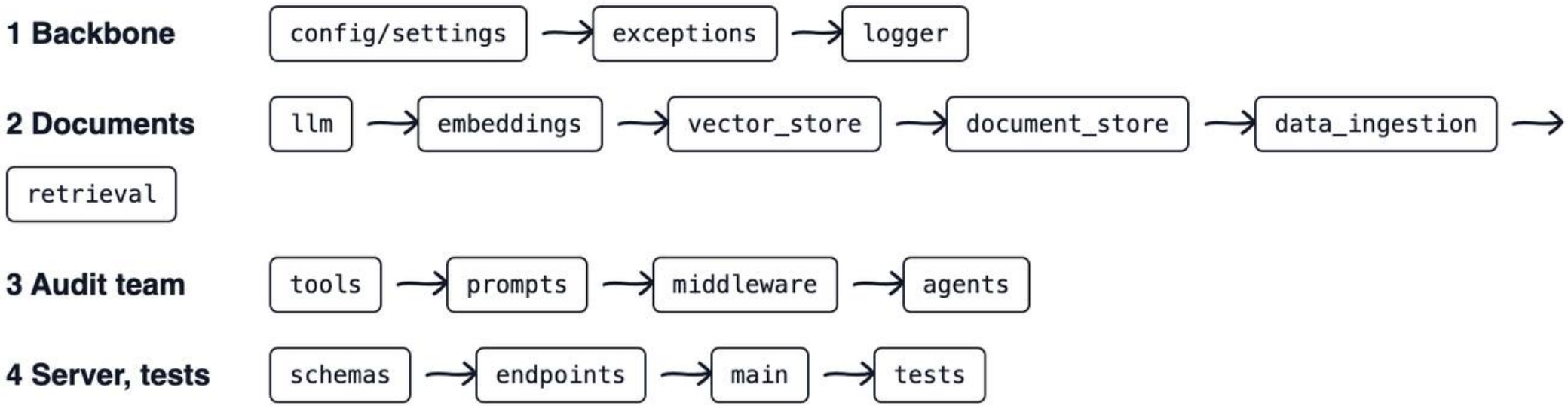


the shared basics: everyone may use them

// 04 STRUCTURE

The build order

Each step uses only what is already built, so every step runs.



// 04 STRUCTURE

Create the folders

run from meridian-ai

```
$ mkdir -p backend/{api,rag,agent,config,logger,exceptions,tests}
$ touch backend/{api,rag,agent,config,logger,exceptions}/__init__.py
$ find backend | sort
```

Windows PowerShell: see the reference sheet

mkdir -p
Creates a folder, and any missing parents.

{a,b,c}
The shell expands it into one folder per name.

__init__.py
An empty file that makes a folder a Python package.

No __init__ in tests/
The test runner does not need one.

// 04 STRUCTURE

Create the empty files

run from meridian-ai

```
$ touch backend/config/settings.py backend/logger/custom_logger.py
$ touch backend/rag/{llm,embeddings,vector_store}.py
$ touch backend/rag/{document_store,data_ingestion,retrieval}.py
$ touch backend/agent/{tools,prompts,middleware,agents}.py
$ touch backend/api/{schemas,endpoints,main}.py
$ touch backend/tests/{conftest,test_api,test_modes_and_middleware}.py
$ find backend -type f | wc -l
24
```

> An empty file is a promise: we will fill each one, in the build order.

24

files, all empty, all in their final places.

// 04 STRUCTURE

Protect secrets and save a first commit

run from meridian-ai

```
$ echo ".env" >> .gitignore
$ echo ".meridian/" >> .gitignore
$ echo "logs/" >> .gitignore
$ echo "node_modules/" >> .gitignore
$ git add -A
$ git commit -m "Project skeleton"
```

> First time with Git? Run git config --global user.name and user.email once.

Four new ignore lines

.env, .meridian/, logs/ and node_modules/ are never saved.

A starting point

If we break something later, we can come back to this.

Check it

git status should say the working tree is clean.

// SUMMARY

The structure in five lines

1. **One file, one job.**
   The name tells you where to look.

2. **Layers point one way.**
   api uses agent and rag; those use config, logger and exceptions.

3. **Twenty-four files, all in place and empty.**
   Created with two mkdir and five touch commands.

4. **A fixed build order.**
   Backbone, document Q&A, audit team, server and tests.

5. **A first commit.**
   With .env ignored from the start.